إعداد عربي تعليمي · الدفعة الأولى

كيف يعمل لينكس

من فهم الصورة العامة إلى ملاحظة النظام بنفسك

استنادًا إلى كتاب

برايان وارد

How Linux Works
What Every Superuser Should Know
Second edition · No Starch Press

مقدمة · الفصل الأول · معجم · مختبر
مسودة تعليمية جزئية، وليست ترجمة حرفية أو رسمية.
الفصول من الثاني إلى السابع عشر لم تُعد بعد.

7 سبتمبر 2026 · Markdown + PDF

محتويات هذه الدفعة

مسار تأسيسي بالعربية: اقرأ الفكرة، شاهد المخطط، ثم جرّب ولاحظ.

مقدمة الطبعة العربية التعليمية	3
الفصل الأول: الصورة العامة	8
قبل أن نبدأ	8
1.1 مستويات التجريد وطبقاته في نظام لينكس	9
1.2 العتاد: فهم الذاكرة الرئيسية	10
1.3 النواة	12
1.4 فضاء المستخدم	17
1.5 المستخدمون	18
1.6 إلى أين نتجه بعد ذلك؟	20
مختبر عملي: شاهد الصورة على جهازك	20
اختبر فهمك	23
خلاصة الفصل	24
معجم الدفعة الأولى	25
ملاحظات الإعداد والمراجع	29
خريطة إنجاز الكتاب	31

العناوين روابط داخلية. الأرقام تشير إلى صفحات هذه النسخة العربية، بما فيها الغلاف. إحالات المصدر في المتن تشير إلى صفحات الملف الإنجليزي. يمكنك كذلك التنقل من العلامات المرجعية في قارئ PDF.

مقدمة الطبعة العربية التعليمية

> **الدفعة الأولى — مسودة إعداد قابلة للمراجعة.** تتضمن هذه الدفعة إعادة صياغة تعليمية لمقدمة الكتاب وفصله الأول، مع معجم وتمارين وإجابات. الفصول من الثاني إلى السابع عشر **لم تُعد بعد.** هذه ليست ترجمة حرفية، ولا نسخة عربية مكتملة من الكتاب.

عن هذا الإعداد

يحمل الكتاب الأصلي عنوان How Linux Works: What Every Superuser Should Know، وهو من تأليف **برايان وارد (Brian Ward)** ونشر **No Starch Press**. يمكن صياغة عنوانه عربيًا هكذا: **كيف يعمل لينكس: ما ينبغي لكل مستخدم ذي صلاحيات عليا أن يعرفه.** يستند هذا المشروع إلى ملف الطبعة الثانية الموجود في المستودع، لا إلى طبعة أحدث.

الهدف هنا أن يصبح فهم لينكس ممكنًا لقارئ عربي يعرف استخدام الحاسوب، لكنه لم يدرس أنظمة التشغيل دراسة متخصصة. نحافظ على تسلسل الأفكار والمفاهيم وأرقام أقسام الفصل، ونشرحها بعربية فصحى واضحة. نضيف أمثلة صغيرة وأسئلة تساعد على الفهم، بدل أن نستبدل كل كلمة إنجليزية بكلمة عربية ونترك المعنى غامضًا.

النص التالي **إعداد تعليمي مستند إلى الكتاب**، وليس كلامًا جديدًا منسوبًا إلى مؤلفه. الفقرات الموسومة «إضافة تعليمية» و«توضيح حديث»، وكذلك المختبر والأسئلة والمعجم، من الإعداد العربي. والرسوم مخططات تعليمية جديدة مستندة إلى أفكار رسوم الفصل، لا نسخ مطابقة لها. تُذكر مصادر التوضيحات التقنية في ملاحظات الإعداد والمراجع.

تعود حقوق النص الأصلي إلى أصحابه. لا يتضمن هذا المشروع ادعاء بأنه ترجمة رسمية أو معتمدة، ولا يمنح بذاته إذنًا بترجمة الكتاب أو إعادة توزيعه؛ ينبغي التحقق من حقوق النشر والتوزيع قبل نشر هذه المادة خارج نطاق العمل عليها.

لماذا نتعلم ما وراء الواجهة؟

الفكرة التي تنطلق منها مقدمة الكتاب بسيطة: من حقك أن تفهم ما يفعله حاسوبك، وأن تستخدم برامجه لتحقيق ما تريد، ضمن حدود قدراتها. ليس المطلوب أن تحفظ مئات الأوامر، بل أن تملك تصورًا يفسر ما يحدث عندما يعمل الجهاز بصورة طبيعية، وما قد يكون حدث عندما يتعطل شيء فيه.

يصلح لينكس لهذا النوع من التعلم لأنه يتيح لك فحص كثير من مكوناته مباشرة. يمكنك قراءة ملفات إعداد نصية، ومشاهدة العمليات الجارية، وتتبع استخدام الموارد، والرجوع إلى توثيق الأدوات. لا يعني ذلك أن كل شيء بسيط، بل إن النظام لا يشترط عليك الاكتفاء بواجهة تخفي تفاصيله.

المشكلة الأولى ليست قلة المعلومات، بل كثرتها: أي ملف يضبط هذه الخدمة؟ من بدأ هذه العملية؟ هل المشكلة في البرنامج أم في الشبكة أم في جهاز التخزين؟ يساعدك فهم العلاقة بين الأجزاء على طرح السؤال الصحيح قبل البحث عن الأمر المناسب.

> **إضافة تعليمية:** عندما تنسخ أمرًا من الإنترنت، اسأل ثلاثة أسئلة: ماذا يقرأ؟ ماذا يغيّر؟ وبصلاحيات مَن يعمل؟ إذا لم تعرف الإجابة، فافهم الأمر أولًا، ولا تجعل صلاحيات المدير وسيلة لتجاوز كل رسالة خطأ.

لمن كُتبت هذه المادة؟

يفيد هذا المسار عدة فئات من القراء:

- مستخدم لينكس الذي يريد الانتقال من تشغيل التطبيقات إلى فهم النظام الذي يشغّلها.
- مسؤول الأنظمة والعامل في التشغيل أو في فرق التطوير والتشغيل، ممن يحتاجون إلى تفسير الأعطال والتعامل مع الخدمات والموارد.
- المطوّر الذي يريد معرفة ما يقدمه نظام التشغيل لبرنامجه، بدل النظر إليه بوصفه صندوقًا مغلقًا.
- الطالب والباحث الذي يدير جهازه بنفسه ويحتاج إلى أساس موثوق للتجربة.
- الهاوي الذي يحب أن يفكك المشكلة، ويغيّر عاملًا واحدًا، ثم يلاحظ النتيجة.

لا يلزم أن تكون مبرمجًا. يكفي أن تعرف معنى الملف والمجلد، وأن تستطيع استخدام واجهة نظامك وفتح تطبيق الطرفية. ستظهر أسماء دوال مثل `fork()`، لكننا سنشرح وظيفتها دون افتراض معرفة بلغة البرمجة C.

ولا يلزم أن تفهم كل التفاصيل من القراءة الأولى. إذا استطعت في نهاية الفصل أن تشرح الفرق بين البرنامج والعملية، وبين المستخدم ذي الصلاحيات العليا والنواة، فقد بنيت أساسًا مهمًا بالفعل.

كيف تقرأ وتجرّب؟

تتدرج فصول الكتاب الأصلي من الأفكار الأساسية إلى التفاصيل. اتبع هذا التدرج: اقرأ الفكرة العامة أولًا، ثم جرّب مثالًا صغيرًا، ثم ارجع إلى التفاصيل التي تجيب عن سؤال ظهر أثناء التجربة. لا تجعل المصطلحات المتقدمة في نهاية موضوع ما تمنعك من متابعة المسار كله.

لكل جلسة تعلم قصيرة، جرّب الخطوات الآتية:

1. اختر مفهومًا واحدًا، مثل «العملية»، واكتب له تعريفًا بكلماتك.
2. توقّع ما سيفعله الأمر قبل تشغيله.
3. شغّله على جهاز تجريبي، واقرأ الناتج بدل الاكتفاء بنجاح التنفيذ.
4. قارن الناتج بتوقعك، وسجّل السؤال الذي بقي بلا إجابة.
5. أعد الشرح لشخص آخر، أو لنفسك كتابةً. إذا تعثرت في الشرح، فأنت تعرف الآن أين تحتاج إلى المراجعة.

يفضّل أن يكون أمامك نظام لينكس تستطيع التجربة عليه. يمكن أن يكون جهازًا مخصصًا للتعلم أو آلة افتراضية. احتفظ بنسخة احتياطية من الملفات المهمة، وبنقطة استعادة للآلة الافتراضية قبل التجارب التي تغيّر إعدادات النظام. هذه وسائل لتقليل المخاطر، وليست ضمانًا ضد كل خطأ.

استخدم حسابًا عاديًا في معظم الوقت. قد تحتاج في فصول لاحقة إلى صلاحيات إدارية، لكن **أوامر مختبر الفصل الأول لا تحتاج إلى** `sudo` **أو إلى حساب** `root`. لا تختبر أوامر إدارية على خادم إنتاج أو جهاز يعتمد عليه الآخرون.

> **توضيح حديث — من الإعداد العربي:** قد تختلف الصورة داخل حاوية أو بيئة مقيدة عنها في آلة افتراضية كاملة. فالحاويات تستخدم عادة نواة النظام المضيف، وقد تعرض مجموعة محدودة من العمليات والموارد. إذا اختلف ناتج أمر عن المثال، افحص طبيعة بيئتك قبل أن تفترض وجود خطأ. راجع مرجع فضاءات الأسماء في قسم المراجع.

قراءة الأوامر دون التباس

الطرفية تطبيق يعرض النص ويسمح بإدخاله، أما **الصدفة** فهي البرنامج الذي يفسّر الأوامر داخلها. الطرفية ليست النواة، والصدفة ليست النواة أيضًا. سنفصل هذه العلاقة أكثر في الفصل الثاني عند إعداده.

يستخدم الأصل الرمز `$` للدلالة على محث حساب عادي، والرمز `#` للدلالة على محث حساب ذي صلاحيات عليا. المحث ليس جزءًا من الأمر الذي تنسخه. وقد يختلف شكله على جهازك؛ لذلك لا تعتمد على شكله وحده لمعرفة صلاحياتك.

في هذا الإعداد نضع الأوامر القابلة للنسخ **من دون المحث**. مثلًا:

```
ls /
```

يعرض هذا الأمر محتويات الدليل الجذر. لا تكتب $ قبله، ولا تستبدل الشرطة المائلة بعلامة أخرى. إذا عرضنا مخرجات، فستأتي في كتلة مستقلة موسومة بأنها مخرجات أو مثال توضيحي، لا أمر مطلوب تشغيله.

نترك أسماء الأوامر والخيارات والمسارات والمتغيرات كما هي: `ls` و`/etc` و`PATH`. تظل كتل الأوامر من اليسار إلى اليمين، حتى داخل الصفحات العربية. أما الشرح المحيط بها فمن اليمين إلى اليسار. والأرقام المستخدمة في الأقسام، مثل `1.3.2`، تتبع ترقيم الأصل لتسهيل المقارنة معه.

قد ترى في المصادر أسماء توثيق مثل `fork(2)` أو `ps(1)`. الرقم بين القوسين هو قسم الدليل، وليس جزءًا من اسم الأمر، ولا رقم إصدار له. سنعرض مراجع واضحة يمكنك الرجوع إليها عندما ترغب في مزيد من التفصيل.

خريطة الكتاب والطبعة التي نعتمدها

يمكن النظر إلى الكتاب بوصفه ثلاث مراحل مترابطة:

- **بناء الصورة الأولى:** التعرف إلى مكونات النظام، ثم أدوات سطر الأوامر وبنية الأدلة.
- **تتبّع النظام من الداخل:** الأجهزة والتخزين والإقلاع والخدمات والإعداد والموارد والشبكات.
- **اكتساب مهارات العمل عليه:** سكربتات الصدفة، ونقل الملفات، وبيئات المستخدم، وسطح المكتب، وأدوات التطوير وبناء البرامج.

يحاول الأصل شرح المبادئ المشتركة بين التوزيعات، مع أمثلة من عائلتي Debian وRed Hat ومشتقاتهما. يركز على الحواسيب المكتبية والخوادم، وتنتقل كثير من الأفكار إلى أنظمة أخرى تستخدم لينكس، لكن أسماء الأدوات والقيود والتفاصيل قد تتغير.

توضح مقدمة الطبعة الثانية أن الكتاب انتقل من التركيز على بعض صعوبات التركيب والإعداد القديمة إلى اهتمام أكبر بدور النواة وإدارة الأجهزة والإقلاع. وهذا وصف للتغييرات بين طبعات الكتاب الأصلي، **لا إعلان عن إصدار عربي جديد مكتمل.**

يحمل الملف المصدر بيانات نشر من عامي 2014 و2015، وتذكر صفحة الحقوق فيه الطبعة الثانية وحقوق عام 2015. لهذا لا نتعامل مع أسماء الخدمات والتوزيعات المذكورة فيه بوصفها وصفًا مضمونًا لجهاز معاصر. نحتفظ بالمفهوم، ونفصل عنه التوضيح الحديث عندما يلزم، بدل تحديث النص بصمت أو نسب معلومات جديدة إلى المؤلف.

اصطلاحات نتفق عليها منذ البداية

نعتمد «لينكس» اسمًا شائعًا في الشرح. وعندما تكون الدقة مهمة، سنميز بين **نواة لينكس** و**التوزيعة** التي تجمع النواة مع الأدوات والمكتبات والخدمات والتطبيقات. تسمية GNU/Linux تبرز دور مشروع GNU في كثير من هذه الأنظمة؛ والتمييز بين المكونات أهم هنا من الخلاف على الاسم المختصر.

نذكر المقابل الإنجليزي للمصطلح الأساسي عند تقديمه، ثم نلتزم بالمقابل العربي قدر الإمكان. نقول «عملية» عند الحديث عن برنامج قيد التنفيذ، و«نواة» عن المكوّن المركزي لنظام التشغيل، و«نواة معالجة» عندما نقصد جزءًا من المعالج. ويجمع المعجم المصطلحات المعتمدة في هذه الدفعة.

ليس الهدف أن تنتهي من القراءة وأنت تعرف كل شيء، بل أن تعرف كيف تسأل وتلاحظ وتتحقق. ابدأ الآن بالفصل الأول: صورة صغيرة واضحة للنظام، قبل الدخول إلى تفاصيل أجزائه.

الفصل الأول: الصورة العامة

> **نطاق الفصل:** إعداد تعليمي للأقسام 1.1 إلى 1.6 من الطبعة الثانية، في صفحات ملف PDF من 8 إلى 14. يحتفظ بالترتيب المفاهيمي للأصل، ويضيف توضيحات موسومة ومختبرًا وأسئلة. أرقام الصفحات هنا هي أرقام صفحات الملف، لا أرقام طبعة ورقية أخرى.

قبل أن نبدأ

عندما تفتح متصفحًا، يبدو أنك تشغّل برنامجًا واحدًا. لكن وراء النافذة عمليات تتبادل البيانات، ومكتبات تقدم وظائف جاهزة، ونواة تنظم الوصول إلى الذاكرة والمعالج والشبكة. وقد يتصل المتصفح بخادم ويب، ويتصل ذلك الخادم بقاعدة بيانات. كيف نفهم هذا العدد من الأجزاء دون أن نضيع في تفاصيله؟

الأداة الأولى هي **التجريد**، أي اختيار مستوى مناسب من التفاصيل. عندما تركب سيارة، لا تحتاج إلى معرفة عدد البراغي التي تثبت محركها. يكفيك أن تعرف أنها تنقلك، وكيف تستعمل الباب وحزام الأمان. أما إذا كنت تقودها، فتحتاج إلى معرفة أدوات التحكم. وإذا أصبحت الرحلة شديدة الاهتزاز، فقد تفكك الصورة إلى ثلاثة احتمالات: السيارة، والطريق، وطريقة القيادة.

لم يغيّر هذا التفكيك السيارة، لكنه غيّر الطريقة التي تنظر بها إلى المشكلة. بدل اتهام المحرك مباشرة، تسأل أولًا: هل الطريق غير مستوٍ؟ وبالمثل، لا يعني بطء موقع ويب بالضرورة أن معالج حاسوبك ضعيف؛ قد يكون السبب في الاتصال أو في الخادم أو في البرنامج نفسه.

يستخدم مطوّرو البرمجيات الفكرة ذاتها. يقسمون النظام إلى **مكوّنات**، لكل منها وظيفة وواجهة للتعامل مع المكوّنات الأخرى. ليس على مطوّر تطبيق أن يعرف كل تفاصيل بطاقة الشبكة، لكنه يحتاج إلى معرفة الواجهة التي يستخدمها لإرسال البيانات.

بعد هذا الفصل، ينبغي أن تستطيع:

- وضع العتاد والنواة وعمليات المستخدم في صورة واحدة.
- التمييز بين الذاكرة الرئيسية والتخزين الدائم، وبين البرنامج والعملية.
- شرح مشاركة المعالج وعزل الذاكرة بصورة أولية.
- وصف ما يحدث عند تشغيل برنامج خارجي من الصدفة.
- التمييز بين نمط تنفيذ النواة وصلاحيات المستخدم `root`.
- ربط هذه الأفكار بملاحظات فعلية من جهاز لينكس، لا بتعريفات محفوظة فقط.

1.1 مستويات التجريد وطبقاته في نظام لينكس

لا يكفي تقسيم النظام إلى أجزاء؛ نحتاج إلى ترتيب يساعدنا على فهم العلاقات بينها. نسمّي مجموعات المكونات **طبقات** أو **مستويات** بحسب قربها من المستخدم أو من العتاد. المتصفح أقرب إلى المستخدم من برنامج تشغيل القرص، وبرنامج تشغيل القرص أقرب إلى العتاد من المتصفح.

في أبسط صورة، يتكون نظام لينكس الجاري من ثلاثة مستويات:

1. **العتاد (Hardware)**: الأجزاء المادية، مثل المعالج والذاكرة والأقراص وواجهات الشبكة. ينفّذ المعالج التعليمات ويعالج البيانات، وتحتفظ الذاكرة بما يحتاج إليه أثناء التنفيذ.
2. **النواة (Kernel)**: برنامج مركزي يدير موارد الجهاز، وينظم استخدام المعالج والذاكرة والأجهزة، ويقدم واجهات للبرامج الأخرى. هي جزء من البرمجيات، وليست قطعة إلكترونية مستقلة.
3. **فضاء المستخدم (User Space)**: البيئة التي تعمل فيها غالبية البرامج والخدمات. البرنامج أثناء تنفيذه يسمى **عملية (Process)**؛ ومجموع عمليات المستخدم يشكل الجزء الذي نتعامل معه عادة، بصورة مباشرة أو غير مباشرة.



الشكل 1-1 — مخطط تعليمي لمستويات النظام، مستند إلى فكرة الشكل في الأصل. ليست الصناديق قائمة بكل مكونات لينكس.

نمط التنفيذ ليس مجرد اسم للطبقة

توجد حدود فعلية يفرضها العتاد بين **نمط المستخدم (User Mode)** و**نمط النواة (Kernel Mode)**. في نمط المستخدم، لا تستطيع التعليمات الوصول بحرية إلى كل الذاكرة أو تنفيذ كل العمليات ذات الامتياز. وعندما تحتاج العملية إلى خدمة محمية، تطلبها من النواة عبر واجهة محددة.

في نمط النواة تتاح صلاحيات منخفضة المستوى أوسع بكثير. وهذا ضروري لإدارة الجهاز، لكنه يعني أيضًا أن خطأً في النواة أو في برنامج تشغيل يعمل ضمنها قد يؤثر في النظام كله. أما خطأ في تطبيق معزول، فكثيرًا ما ينتهي بتوقف تلك العملية وحدها بدل توقف بقية البرامج.

احفظ الفرق بين اصطلاحين متقاربين:

- **النمط** يصف مستوى الامتياز الذي ينفذ به المعالج التعليمات.
- **الفضاء** يشير هنا إلى مجال الذاكرة والبيئة التي تعمل فيها النواة أو عمليات المستخدم.

يُسمّى المجال المحمي المخصص للنواة **فضاء النواة (Kernel Space)**. أما فضاء المستخدم فليس ذاكرة واحدة مشتركة يستطيع أي تطبيق العبث بكل ما فيها؛ لكل عملية عادة مساحة عناوين خاصة بها، كما سنرى في القسم 1.3.2.

الحماية لا تلغي كل المخاطر

إذا تعطل المتصفح، يفترض ألا يتسبب ذلك وحده في إتلاف ذاكرة عملية حسابية تعمل في الخلفية. لكن عزل الذاكرة لا يحمي ملفاتك من برنامج سمحت له بتعديلها. عملية تعمل باسمك قد تحذف ملفات تملكها، وعملية ذات صلاحيات واسعة قد تُحدث ضررًا أوسع.

> **إضافة تعليمية:** هناك فرق بين «لا يستطيع البرنامج الكتابة مباشرة في ذاكرة النواة» و«لا يستطيع البرنامج إيذائي». قد يكون ممنوعًا من الأولى ومسموحًا له بتغيير بياناتك. الصلاحيات والنسخ الاحتياطية والتحديثات تكمل بعضها؛ ولا يغني واحد منها عن البقية.

1.2 العتاد: فهم الذاكرة الرئيسية

الذاكرة الرئيسية، التي نرمز إليها عادة بـRAM، مساحة سريعة نسبيًا للاحتفاظ بالتعليمات والبيانات التي يحتاج إليها النظام أثناء عمله. على مستوى مبسط جدًا، تحتوي على أنماط من الأصفار والآحاد. يسمى كل صفر أو واحد بتًا **(Bit)**، وتتكون البايتات المعتادة من ثمانية بتات.

النواة والبرامج والبيانات ليست أشياء سحرية داخل الجهاز؛ إنها تمثيلات بتية تعطيها طريقة الاستخدام معناها. قد يمثل نمط ما جزءًا من تعليمة، أو حرفًا، أو عددًا، أو لونًا في صورة. يتعامل المعالج مع التعليمات والبيانات ويغيّر حالة النظام بتنفيذ العمليات عليها.

يفرق هذا التصور بين **الذاكرة الرئيسية والتخزين الدائم**. ملفاتك على قرص أو وحدة SSD يمكن أن تبقى بعد انقطاع الطاقة، أما محتوى ذاكرة RAM المعتادة فيضيع عند انقطاعها. عندما تشغّل برنامجًا، يُحمّل ما يلزم من تعليماته وبياناته إلى الذاكرة، ولا يتحول القرص نفسه إلى معالج.

> **إضافة تعليمية:** تشبيه الذاكرة بسطح مكتب والتخزين بخزانة مفيد كبداية: تضع على المكتب ما تعمل عليه الآن، وتحتفظ في الخزانة بما تريد بقاءه. لكنه تشبيه محدود؛ فالأنظمة تستخدم التخزين المؤقت وخرائط الذاكرة وآليات أخرى، ولا تنسخ بالضرورة البرنامج كله دفعة واحدة.

ماذا نعني بحالة النظام؟

الحالة (State) وصف للنظام أو لأحد مكوناته في لحظة معينة. إذا كان لدينا أربعة بتات فقط، فكل ترتيب من الترتيبات الآتية يمثل حالة مختلفة:

```
0110
0001
1011
```

لكن وصف ملايين البتات واحدة واحدة لا يساعدنا عادة. نرفع مستوى التجريد فنقول: «العملية تنتظر إدخالًا»، أو «البرنامج يقرأ ملفًا»، أو «هذه الصفحة من الذاكرة للقراءة فقط». هذه أوصاف مفيدة لحالات لها تمثيل أدق داخل الجهاز.

قد يظهر أيضًا مصطلح **صورة العملية (Process Image)**. لا يعني صورة مرئية على الشاشة، بل تمثيل البرنامج وبياناته في الذاكرة. وسيساعدنا هذا المصطلح عند الحديث عن استبدال البرنامج الجاري بواسطة عائلة exec.

> **توضيح تقني — من الإعداد العربي:** لا يقرأ المعالج كل معلومة من شرائح RAM مباشرة في كل خطوة؛ توجد سجلات وذاكرات مخبئية وآليات لنقل البيانات. نتجاوز هذه التفاصيل الآن لأن المقصود بناء نموذج للذاكرة والعمليات، لا شرح البنية الدقيقة للمعالج.

1.3 النواة

تدير النواة موارد تتشاركها عمليات كثيرة. عليها أن تعرف أي عملية تستطيع استخدام المعالج، وأي صفحات ذاكرة مخصصة لها، وكيف تنفذ طلبًا إلى جهاز، وما إذا كان الطلب مسموحًا أصلًا.

يمكن جمع مسؤولياتها في أربعة مجالات:

المجال	السؤال الذي تجيب عنه النواة
إدارة العمليات	مَن يعمل الآن؟ ومَن ينتظر؟ وكيف نبدأ العمل أو ننهيه؟
إدارة الذاكرة	لمن تخص هذه الذاكرة؟ وما الذي يجوز مشاركته أو الوصول إليه؟
برامج تشغيل الأجهزة	كيف نطلب من عتاد معين تنفيذ عملية بطريقة مضبوطة؟
نداءات النظام والخدمات المساندة	كيف تحصل عمليات المستخدم على خدمات لا تستطيع تنفيذها مباشرة؟

هذه حدود تعليمية، وليست جدرانًا بين وظائف مستقلة تمامًا. قراءة ملف مثلًا قد تجمع بين إدارة الذاكرة، وجدولة عملية تنتظر، والتواصل مع جهاز تخزين.

1.3.1 إدارة العمليات

البرنامج تعليمات محفوظة، غالبًا في ملف. أما **العملية** فهي مثيل جارٍ من التنفيذ، له معرّف وذاكرة وحالة وموارد مرتبطة به. يمكنك تشغيل البرنامج نفسه أكثر من مرة فتظهر عمليات مختلفة، وقد يستخدم تطبيق واحد عدة عمليات.

تشمل إدارة العمليات إنشاءها، وجدولتها، وإيقافها مؤقتًا عند الحاجة، واستئنافها، وإنهاءها. الجزء الأقل وضوحًا هو مشاركة المعالج: كيف يبدو أن المتصفح ومشغل الصوت ومحرر النصوص تعمل جميعًا في الوقت نفسه؟

لنتخيل أولًا معالجًا منطقيًا واحدًا. لا تنفذ عليه مهمتان مستقلتان في اللحظة نفسها، لكن النواة تمنح كل مهمة فرصة للعمل، ثم قد تنتقل إلى أخرى. يكون التناوب سريعًا بحيث نرى النظام متعدد المهام، لا جهازًا يتوقف بعد كل تعليمة ليستأذننا.

تسمى الفترة التي نتصور أن عملية حصلت فيها على المعالج **شريحة زمنية (Time Slice)**. ويسمى الانتقال من سياق تنفيذ إلى سياق آخر **تبديل السياق (Context Switch)**. والسياق هو المعلومات اللازمة لمتابعة التنفيذ من الموضع الصحيح، ومنها قيم سجلات المعالج وحالة الجدولة ومعلومات مساحة العناوين.

> **إضافة تعليمية:** تخيل مدرسًا يساعد عدة طلاب بالتناوب. لا يجيب الجميع في اللحظة نفسها، لكنه يتذكر أين توقف مع كل طالب. هذا يشرح التزامن، لا سرعات المعالجات ولا الخوارزمية الحقيقية للجدولة.

صورة مبسطة لتبديل السياق

في سيناريو تعليمي تنتهي فيه فرصة عملية لاستخدام المعالج، يمكن تلخيص ما يحدث هكذا:

1. تحدث مقاطعة من المؤقت، فينتقل التنفيذ إلى النواة وتستعيد التحكم على ذلك المعالج.
2. تحفظ النواة حالة التنفيذ اللازمة لاستئناف العملية لاحقًا. لا يعني هذا نسخ محتويات ذاكرة العملية كلها عند كل تبديل.
3. تعالج ما يلزم من أعمال مرتبطة بالمقاطعة أو بأحداث أصبحت جاهزة.
4. يفحص المجدول المهام القابلة للتنفيذ ويختار المهمة التالية وفق سياسة الجدولة.
5. تجهّز النواة سياق المهمة المختارة وما يلزم لاستخدام مساحة عناوينها.
6. تُضبط آلية متابعة الوقت والجدولة بحسب تنفيذ النظام.
7. يعود التنفيذ إلى نمط المستخدم عندما تصبح العملية المختارة جاهزة للمتابعة فيه.

هذه **صورة تفسيرية**، لا تسلسلًا إلزاميًا ثابتًا لكل حالة. قد تترك العملية المعالج لأنها تنتظر بيانات من الشبكة قبل انتهاء أي فرصة زمنية متوقعة. وقد يختار المجدول العملية نفسها مجددًا إذا لم توجد مهمة أخرى أنسب للتنفيذ.

التزامن والتوازي

التزامن (Concurrency) يعني أن عدة أعمال تتقدم خلال الفترة نفسها، ولو بالتناوب. أما **التوازي (Parallelism)** فيعني تنفيذ أعمال فعلًا في اللحظة نفسها على موارد تنفيذ متعددة. على جهاز متعدد الأنوية يمكن أن يحدث الأمران معًا: عمل متوازٍ بين الأنوية، وتناوب بين مهام أكثر من عدد المعالجات المنطقية المتاحة.

> **توضيح حديث — من الإعداد العربي:** تستخدم الفقرة السابقة كلمة «عملية» لتبسيط الفكرة. فعليًا، تحتوي العملية على خيط تنفيذ واحد أو أكثر، وتُجدول خيوط التنفيذ بوصفها مهام قابلة للتنفيذ. تشترك خيوط العملية في مساحة عناوينها، وقد تعمل على معالجات مختلفة. ليس عدد العمليات مساويًا لعدد الأنوية. انظر مرجع `pthreads(7)`.

ولا تعمل النواة فقط بين الشرائح الزمنية. تُنفّذ تعليمات منها أيضًا لمعالجة نداءات النظام والمقاطعات والاستثناءات، مثل أخطاء الصفحات. كذلك لا يعني كل انتقال إلى نمط النواة حدوث تبديل إلى عملية أخرى؛ فقد تطلب العملية خدمة وتعود النواة إلى العملية ذاتها.

1.3.2 إدارة الذاكرة

تحتاج النواة إلى تنظيم الذاكرة بحيث تتحقق متطلبات تبدو متعارضة:

- يبقى لها مجال محمي لا تستطيع عمليات المستخدم الوصول إليه اعتباطًا.
- تحصل كل عملية على مساحة للعمل، من دون أن تكتب مباشرة في الذاكرة الخاصة بعملية أخرى.
- تستطيع عمليات مشاركة مناطق محددة عندما يكون ذلك مقصودًا ومسموحًا.
- يمكن حماية بعض المناطق من الكتابة، مثل صفحات تحتوي على تعليمات ينبغي ألا تتغير.
- يمكن استخدام التخزين المساند لبعض البيانات عند تفعيل آليات المبادلة، بدل اشتراط وجود كل محتوى الذاكرة الافتراضية في RAM طوال الوقت.

يساعد العتاد على ذلك بواسطة **وحدة إدارة الذاكرة (MMU)**. في نموذج لينكس المعتاد على الحواسيب العامة، تتعامل العملية مع **عناوين افتراضية**، لا مع أرقام مواقع مادية حرة في شرائح الذاكرة. تنظم النواة الخرائط وحقوق الوصول، وتستخدم وحدة إدارة الذاكرة هذه المعلومات عند ترجمة العناوين والتحقق من الوصول.

يمكن أن تستخدم عمليتان العنوان الافتراضي نفسه، لكنه يشير في كل منهما إلى موضع مادي مختلف. ولهذا لا يكفي أن تعرف عملية عنوانًا في برنامج آخر كي تصبح قادرة على قراءة بياناته.

فكر في المثال الآتي بوصفه رسمًا للمفهوم، لا قيمًا مأخوذة من جهاز حقيقي:

العملية	العنوان الذي تستخدمه	نتيجة الترجمة
العملية أ	`0x1000`	موضع مادي مخصص للعملية أ
العملية ب	`0x1000`	موضع مادي آخر مخصص للعملية ب

تسمى البنى المستخدمة لتنظيم هذه الخرائط **جداول الصفحات (Page Tables)**. تدير النواة هذه الجداول، ويتعاون العتاد معها على تطبيقها. وقد تشير خرائط عمليتين إلى الصفحة المادية نفسها إذا كانت المشاركة مقصودة، مثل بعض الصفحات المشتركة للقراءة فقط.

> **توضيح تقني — من الإعداد العربي:** الذاكرة الافتراضية ليست مرادفًا لمساحة المبادلة swap. يظل لكل عملية فضاء عناوين افتراضي حتى على جهاز بلا مساحة مبادلة. كما أن وجود مساحة افتراضية كبيرة لا يضمن وجود ذاكرة فعلية كافية لكل طلب؛ قد يفشل تخصيص الذاكرة أو يتعرض النظام لضغط شديد. سنحتاج إلى هذا التمييز عند دراسة الموارد في الفصل الثامن.

المطلوب الآن فهم العلاقة: **العملية تستخدم عناوين؛ النواة تدير الخرائط والسياسات؛ والعتاد يساعد على الترجمة والحماية.** لا تحتاج بعد إلى حفظ بنية جدول الصفحات أو أحجام الصفحات المختلفة.

1.3.3 برامج تشغيل الأجهزة وإدارتها

قد تؤدي بطاقتا شبكة الوظيفة نفسها للمستخدم، لكن تختلف الأوامر التي يفهمها عتاد كل منهما. لو اضطر كل تطبيق إلى دعم كل بطاقة وكل قرص مباشرة، لأصبح تطوير البرامج وصيانتها أصعب بكثير.

لهذا توجد **برامج تشغيل الأجهزة (Device Drivers)**. يعرف برنامج التشغيل تفاصيل التعامل مع جهاز أو فئة من الأجهزة، ويساعد على تقديم واجهة منتظمة لبقية النظام. يطلب التطبيق عملية أعلى مستوى، وتتولى طبقات النظام إيصال الطلب إلى العتاد المناسب.

ينتمي كثير من برامج التشغيل إلى النواة، سواء أكان مدمجًا فيها أم محمّلًا بوصفه وحدة. ويسمح هذا الموقع بالوصول المضبوط إلى موارد لا ينبغي أن تكون متاحة مباشرة لأي عملية. فالتعامل الخاطئ مع جهاز تخزين أو مع عمليات الوصول المباشر إلى الذاكرة قد يؤثر في بيانات غير البرنامج الذي بدأ الطلب.

> **إضافة تعليمية:** لا تفهم عبارة «برنامج تشغيل» بمعنى تطبيق تفتحه من قائمة البرامج. وليس كل برنامج يتعامل مع جهاز جزءًا من النواة؛ توجد أدوات ومكتبات وخدمات في فضاء المستخدم تتعاون مع برامج التشغيل. تفاصيل هذا التعاون تأتي في فصل الأجهزة.

الفكرة المهمة أن البرنامج لا يحتاج عادة إلى معرفة الشركة المصنعة للقرص لكي يقرأ ملفًا. يحصل على واجهة مناسبة، بينما تختبئ تفاصيل كثيرة وراءها. هذا هو التجريد نفسه الذي بدأنا به الفصل، ولكن في مسار عملي داخل النظام.

1.3.4 نداءات النظام والخدمات المساندة

كيف تطلب عملية في نمط المستخدم خدمة من النواة؟ تستخدم **نداء نظام (System Call)**، مباشرة أو عبر دالة في مكتبة. فتح ملف، وقراءة بيانات، وكتابة بيانات، وبدء عملية أو انتظارها أمثلة على أعمال تتضمن التواصل مع النواة.

ليس نداء النظام أمرًا تكتبه بالضرورة في الطرفية. عندما تنفذ أمرًا مثل `ls`، قد يستخدم البرنامج عدة نداءات نظام أثناء عمله. القوسان في أسماء مثل `fork()` أسلوب شائع لكتابة أسماء الدوال؛ لا يعنيان أنك مطالب بإدخال الاسم وحده في الصدفة.

إنشاء عملية ليس هو استبدال برنامجها

لفهم تشغيل البرامج، نحتاج إلى التمييز بين عمليتين مفاهيميتين:

- `fork()` — **إنشاء عملية ابنة:** تنشئ عملية موجودة عملية جديدة تشبهها عند البداية. تسمى الأولى العملية الأم، والثانية العملية الابنة. يحصل الابنة على معرّف عملية خاص به، وله مساحة عناوين مستقلة من حيث المعنى الذي يراه البرنامج.
- عائلة `exec` — **استبدال البرنامج الجاري:** تستبدل العملية برنامجها الحالي ببرنامج آخر. لا تُنشأ بسبب هذا الاستبدال وحده عملية جديدة ذات معرّف جديد؛ يستمر معرّف العملية بينما تتغير صورة البرنامج وذاكرته المرتبطة به.

> **توضيح تقني — من الإعداد العربي:** يعبّر الاسم المختصر `exec()` في الشرح عن عائلة واجهات، وليس اسم نداء وحيد يكفي لاستدعائه حرفيًا. تنتهي واجهات شائعة من هذه العائلة إلى نداء مثل `execve()`. وعند نجاح الاستبدال لا يعود التنفيذ إلى البرنامج السابق بعد النداء. انظر مرجع `execve(2)`.

لنفترض أنك تستخدم صدفة تفاعلية، ثم تشغّل البرنامج الخارجي الآتي:

```
ls /
```

في النموذج المعتاد المبسط، تنشئ الصدفة عملية ابنة، ثم يستبدل الابنة صورته ببرنامج `ls`. تعرض العملية الجديدة محتويات الدليل وتنتهي، بينما تبقى الصدفة الأم لتستقبل أمرًا آخر. قد تنتظر الصدفة انتهاء الأمر الأمامي قبل أن تعرض محثها من جديد.

قبل التشغيل
صدفة أم — معرّف افتراضي 1200

`fork()` ينشئ عملية ابنة

العملية الأم	العملية الابنة
تبقى صدفة، ومعرّفها 1200	تبدأ كنسخة من الصدفة، بمعرّف 1201
يمكنها انتظار انتهاء الابنة	تستخدم إحدى واجهات exec لتشغيل ls
تعرض المحث مجددًا بعد انتهاء الأمر الأمامي	تبقى بالمعرّف 1201، ثم تنتهي بعد إنجاز العمل

الشكل 2-1 — مخطط تعليمي لإنشاء عملية ابنة ثم استبدال برنامجها. الأرقام للتوضيح فقط؛ لا تستخدمها لاستهداف عمليات على جهازك.

كلمة «نسخة» في شرح `fork()` لا تعني نسخ كل صفحات الذاكرة المادية فورًا. يستخدم لينكس **النسخ عند الكتابة (Copy-on-Write)**: يمكن أن تتشارك الصفحات المادية في البداية، وتُنشأ نسخة مستقلة عند الحاجة إلى تعديل صفحة خاصة مشتركة بهذه الطريقة. تظل العزلة المنطقية بين فضاءي العناوين قائمة. يشرح مرجع `fork(2)` هذه الآلية.

هناك أيضًا حدود لهذا النموذج التعليمي. بعض أوامر الصدفة مدمج فيها، فلا تحتاج إلى تشغيل برنامج خارجي جديد. وتوجد واجهات وآليات مثل `clone()` و`clone3()` تتحكم في إنشاء المهام وما تتشاركه؛ لذلك لا نحول مثال `fork` ثم `exec` إلى قاعدة حرفية تصف كل مسار تنفيذ على لينكس. تظل الفكرة التي يجب تذكرها صحيحة: **الإنشاء يضيف عملية، والاستبدال يغيّر البرنامج داخل عملية موجودة.**

توجد عند بدء النظام عملية أولى في فضاء المستخدم، تؤدي دور التهيئة وتكون ذات المعرّف `1` في فضاء معرّفات العمليات الرئيسي. يقدمها الأصل باسم `init`، وسنعود إلى بدء فضاء المستخدم في الفصل السادس. لا تستنتج من هذا الاسم أن كل جهاز يستخدم البرنامج نفسه لتنفيذ هذا الدور.

أجهزة موجودة بوصفها واجهات برمجية

تقدم النواة أيضًا **أجهزة زائفة (Pseudo-devices)**: واجهات تبدو للبرنامج شبيهة بملفات الأجهزة، لكنها لا تقابل بالضرورة قطعة عتاد مستقلة. من أمثلتها `/dev/random`، الذي يتيح الوصول إلى بيانات من مولّد الأرقام العشوائية في النواة.

يسمح هذا التصميم بإعادة استخدام واجهات مألوفة، مثل فتح ملف والقراءة منه، للوصول إلى وظيفة يقدمها النظام. وكون الجهاز زائفًا لا يعني تجاوز النواة؛ ففتح واجهته وقراءتها يظلان يتضمنان نداءات نظام.

> **إضافة تعليمية:** المثال هنا لشرح الواجهة، وليس وصفة لاختيار مصدر عشوائية لتطبيق أمني. لا تصمم التشفير أو توليد المفاتيح بنفسك اعتمادًا على هذا الوصف المختصر؛ استخدم واجهات ومكتبات موثقة مناسبة للمهمة.

1.4 فضاء المستخدم

معظم العمل الذي تراه يحدث في فضاء المستخدم: تحرير النصوص، وتصفح الويب، وخدمة قواعد البيانات، وإدارة جلسة سطح المكتب. وقد تسمع الاسم الإنجليزي `userland` للدلالة على البيئة نفسها بصورة غير رسمية.

لا تعني **عملية مستخدم** بالضرورة أن شخصًا يتفاعل معها الآن. خادم الويب الذي يعمل في الخلفية عملية مستخدم أيضًا. ولا تعني العبارة أنها تعمل باسم مستخدم عادي؛ يمكن أن تعمل خدمة باسم `root` وتبقى عملية في فضاء المستخدم.

يمكن ترتيب مكونات هذه البيئة، لأغراض الفهم، في مستويات خدمة تقريبية. توجد أدوات صغيرة تؤدي وظائف محددة، وخدمات أكبر تعتمد عليها، وتطبيقات يتعامل معها المستخدم مباشرة.



الشكل 3-1 — ترتيب تعليمي تقريبي لخدمات فضاء المستخدم. جميع الصناديق هنا في فضاء المستخدم، بما فيها الصندوق السفلي.

هذا الرسم ليس قانونًا يفرضه النظام. قد يكتب برنامج سجلاته بنفسه، أو يرسلها إلى خدمة تسجيل مشتركة. ويمكن اعتبار خادم قاعدة البيانات تطبيقًا عالي المستوى لأن عمله معقد، أو خدمة وسطية لأن تطبيقًا آخر يعتمد عليه. التصنيف يتغير بحسب السؤال الذي نحاول الإجابة عنه.

يشير الأصل إلى `syslog` مثالًا على خدمة تسجيل مشتركة. **ومن التوضيحات المضافة:** قد ترى أيضًا `systemd-journald` في أنظمة تستخدم منظومة `systemd`. ليس المطلوب الآن اختيار خدمة تسجيل، بل فهم أن مكوّنًا يستطيع تفويض وظيفة إلى مكوّن آخر. وتظل تفاصيل النظام الفعلية موضوعًا للفحص، لا استنتاجًا من الرسم وحده.

> **إضافة تعليمية:** الفصل بين طبقات الخدمة في هذا القسم تنظيمي ووصفي. أما الحد بين نمط المستخدم ونمط النواة في القسم 1.1 فهو حد امتياز يفرضه العتاد والنظام. لا تخلط بين نوعي «الطبقات».

1.5 المستخدمون

في لينكس، **المستخدم (User)** هوية يمكن أن تُنسب إليها عمليات وملفات وصلاحيات. قد تمثل شخصًا حقيقيًا، وقد تكون حسابًا مخصصًا لخدمة. وجود حساب على النظام لا يعني بالضرورة وجود شخص يجلس أمام شاشة ويسجل الدخول به.

تتعامل عادة مع اسم مفهوم، مثل `amina`. أما النواة فتستند في كثير من قرارات الصلاحيات إلى معرّفات رقمية، من أهمها **معرّف المستخدم (UID)**. توجد آليات في فضاء المستخدم تربط الاسم بالرقم؛ وسنعود إلى تفاصيل حسابات المستخدمين في الفصل السابع.

لعمليات المستخدم بيانات اعتماد تحدد الهوية والصلاحيات التي تعمل بها. وفي النموذج الأساسي، يستطيع المستخدم إدارة عملياته والوصول إلى ملفاته وفق الأذونات، ولا يملك حرية التصرف في عمليات الآخرين وملفاتهم. توجد تفاصيل واستثناءات، لكن الحدود بين الحسابات أحد أسباب وجود مفهوم المستخدم أصلًا.

المستخدم ذو الصلاحيات العليا

يمثل الحساب `root`، المرتبط تقليديًا بالمعرّف `0`، المستخدم ذا الصلاحيات العليا **(Superuser)**. تسمح امتيازاته الإدارية بأعمال تتجاوز ما يسمح به للحساب العادي، مثل إدارة كثير من عمليات المستخدمين وإعدادات النظام.

لكن هناك حقيقة لا تتغير بسبب قوة الحساب:

البرنامج الذي يعمل باسم `root` لا يصبح هو النواة، ولا ينفذ كل تعليماته في نمط النواة. يبقى برنامجًا في فضاء المستخدم، ويطلب الخدمات من النواة، لكنها تسمح له بطلبات أكثر بناءً على بيانات اعتماده والسياسات المطبقة.

> **توضيح حديث — من الإعداد العربي:** لا تختزل صلاحيات لينكس المعاصر في عبارة «`root` يستطيع كل شيء». تقسم آلية القدرات `capabilities` امتيازات واسعة إلى صلاحيات أصغر، وقد تضيف فضاءات الأسماء وسياسات الحماية وقيود أخرى حدودًا إضافية. والحساب المسمى `root` داخل بيئة معزولة ليس بالضرورة مكافئًا لصلاحيات مدير النظام المضيف. راجع مرجعي القدرات وفضاءات الأسماء.

استخدم أقل صلاحيات تحقق المهمة. الخطأ في اسم مسار أثناء العمل بصلاحيات واسعة قد يُنفّذ بنجاح بدل أن يمنعه النظام. لهذا نفضل التعلم بحساب عادي، ولا نضيف `sudo` إلى أمر لمجرد أن نتيجته لم توافق توقعاتنا.

ولا تخلط بين اسم المستخدم `root` و**الدليل الجذر** `/`. الأول حساب وهوية، والثاني نقطة بداية شجرة الملفات. تشابه التسمية لا يجعلهما الشيء نفسه.

المجموعات

المجموعة (Group) وسيلة لجمع مستخدمين ضمن هوية مشتركة لأغراض الصلاحيات. بدل السماح لكل مستخدم منفردًا بالوصول إلى ملف تعاوني، يمكن منح الإذن لمجموعة مناسبة وإضافة المستخدمين إليها.

ترتبط المجموعات أيضًا بمعرّفات رقمية تسمى GID. وقد ينتمي المستخدم إلى أكثر من مجموعة. سنرى تفاصيل ملكية الملفات وأذونات المجموعات لاحقًا؛ يكفي الآن أن تفهم أنها أداة لتنظيم المشاركة، لا إلغاء الحدود بين الحسابات.

1.6 إلى أين نتجه بعد ذلك؟

لدينا الآن نموذج أولي لنظام يعمل: عتاد يقدم موارد، ونواة تنظمها، وعمليات في فضاء المستخدم تنجز الأعمال. توجد النواة وأجزاء التنفيذ اللازمة للعمليات في الذاكرة، وتؤثر هويات المستخدمين في ما يجوز للعمليات طلبه.

بقي عنصر مهم لم نفصله: أين تبقى البرامج والبيانات عندما لا تكون قيد التنفيذ؟ هنا تأتي الملفات والأدلة والتخزين الدائم. ينتقل الفصل الثاني من الأصل إلى الأدوات الأساسية للتعامل مع هذه البيئة، قبل التعمق في الأجهزة والأقراص والإقلاع.

الفصل الثاني غير متاح في هذه الدفعة. قبل الانتقال إليه عند إعداده، استخدم المختبر الآتي لتحويل مفاهيم الفصل الأول إلى أشياء تستطيع ملاحظتها. لا تنتظر أن تفهم كل خيارات الأوامر؛ سنشرح وظيفة الخيارات المستخدمة هنا فقط.

مختبر عملي: شاهد الصورة على جهازك

> **إضافة تعليمية بالكامل.** هذه التجارب من الإعداد العربي وليست ترجمة لتمارين في الأصل. شغّلها بحساب عادي على نظام لينكس تجريبي. لا تغيّر إعدادات النظام ولا تحتاج إلى صلاحيات إدارية. تحتاج أوامر ps وfree إلى أدوات procps المعتادة؛ قد تكون غائبة أو مختلفة في الأنظمة المصغرة والحاويات. إذا لم يوجد أمر، اقرأ رسالة الخطأ ولا تحاول إصلاح ذلك بصلاحيات واسعة دون فهم.

التجربة الأولى: النواة ليست التوزيعة

اعرض إصدار النواة:

```
uname -r
```

ثم اقرأ معلومات التوزيعة إن كان الملف موجودًا:

```
cat /etc/os-release
```

الأمر الأول يسأل عن إصدار النواة الجارية. والثاني يقرأ ملفًا نصيًا يصف نظام التشغيل أو التوزيعة في بيئتك. قد تستخدم توزيعتان مختلفتان إصدار نواة متقاربًا، وقد تُحدَّث النواة دون أن يتغير اسم التوزيعة.

سؤال الملاحظة: هل ظهر الاسم الذي تستعمله لتوزيعتك في ناتج الأمر الأول أم الثاني؟ وإذا كنت داخل حاوية، فهل من المعقول أن تصف النتيجتان جانبين مختلفين من البيئة؟

التجربة الثانية: العمليات ومعرّفاتها

اعرض بضعة أسطر من قائمة العمليات:

```
ps -eo pid,ppid,user,comm | head -n 8
```

تعني `-e` اختيار كل العمليات التي تستطيع الأداة رؤيتها، وتحدد `-o` الأعمدة المطلوبة. يحمل العمود `PID` معرّف العملية، و`PPID` معرّف العملية الأم، و`USER` المستخدم، و`COMMAND` اسم الأمر. تنقل العلامة `|` الناتج إلى `head` ليعرض أول ثمانية أسطر، ومنها سطر العناوين.

القائمة لقطة لحظة التنفيذ، وليست شاشة متابعة مستمرة. قد تظهر فيها أداة `ps` نفسها، وقد تختلف العمليات والأرقام في كل مرة.

انظر الآن إلى العملية ذات المعرّف `1` في بيئتك:

```
ps -p 1 -o pid,comm,args
```

تختار `-p 1` ذلك المعرّف، ويضيف `args` سطر الأمر بما فيه وسائطه. قد ترى `systemd` أو برنامجًا آخر، وفي الحاوية قد ترى برنامج الحاوية نفسه. سجّل ما رأيت فعلًا، ولا تغيّر الإعداد لتجعل الناتج يطابق اسمًا في كتاب.

التجربة الثالثة: عملية جديدة دون تغيير ملفات

الصق الكتلة التالية كاملة في صدفة تدعم صياغة الصدفة التقليدية، مثل `bash` أو `dash`:

```
sleep 5 &
demo_pid=$!
ps -o pid,ppid,user,stat,comm -p "$demo_pid"
wait "$demo_pid"
```

- ينشئ `sleep 5` برنامجًا ينتظر نحو خمس ثوانٍ ثم ينتهي.
- تجعل `&` الأمر يعمل في الخلفية، حتى تستطيع الصدفة متابعة السطر التالي.

- تحفظ `$!` معرّف آخر عملية شغّلتها الصدفة في الخلفية، ونضعه في المتغير `demo_pid`.
- يعرض `ps` معلومات هذه العملية، ومنها `STAT`، أي مؤشرات حالة العملية.
- ينتظر `wait` انتهاء العملية الابنة. لا تبقى هذه التجربة تعمل إلى أجل غير محدد.

قد ترى حالة تبدأ بالحرف `S` لأن العملية نائمة انتظارًا لانقضاء المدة، لا لأنها تستهلك المعالج في عدّ متواصل. إذا تأخرت بين الأسطر، فقد تنتهي العملية قبل أن يعرضها `ps`، فتظهر العناوين فقط. هذا سلوك متوقع، لا تلف في النظام.

سؤال الملاحظة: لماذا يختلف معرّف هذه العملية عن معرّف الصدفة التي بدأتها؟ وما الذي يدل عليه عمود العملية الأم؟

التجربة الرابعة: الذاكرة والمبادلة

```
free -h
```

تعرض `-h` القيم بوحدات أسهل للقراءة. ابحث عن سطر الذاكرة وسطر المبادلة، ثم عن الأعمدة التي تعرضها نسختك من الأداة. إذا وجدت عمود `available`، فهو تقدير للذاكرة المتاحة لبدء أعمال جديدة دون الحاجة إلى المبادلة، وليس مرادفًا مباشرًا لكل الذاكرة غير المستخدمة الآن.

قد تكون مساحة المبادلة صفرًا مع استمرار النظام في استخدام الذاكرة الافتراضية؛ لا تعارض بين الأمرين. ولا تفترض أن الأرقام داخل حاوية تصف دائمًا حد الذاكرة المسموح به للحاوية نفسها؛ قد تحتاج حدود الموارد إلى أدوات وفحص إضافيين في فصل الموارد.

التجربة الخامسة: مَن أنت بالنسبة إلى النظام؟

```
id
```

يعرض الأمر معرّف المستخدم والمجموعة والمجموعات الإضافية. مثال توضيحي، **وليس ناتجًا مطلوبًا على جهازك**:

```
uid=1000(amina) gid=1000(amina) groups=1000(amina),27(sudo)
```

الأرقام والأسماء تختلف بين الأنظمة. العضوية في مجموعة اسمها `sudo` لا تعني أن كل عملية تشغلها تعمل تلقائيًا باسم `root`؛ السماح برفع الصلاحيات تحدده سياسة النظام، ويتم استخدامه عند طلب العملية الإدارية المناسبة.

لا تنشر نواتج تحتوي على معلومات شخصية أو أسماء أجهزة أو وسائط أوامر حساسة دون مراجعتها. تكفي ملاحظاتك المحلية لإنجاز هذه التجارب.

اختبر فهمك

حاول الإجابة دون الرجوع إلى الفقرات، ثم قارن شرحك بالإجابات المقترحة.

1. لماذا لا يكون تعطل المتصفح دليلًا على تعطل النواة؟ وهل يحمي عزل الذاكرة ملفاتك من كل برنامج؟
2. هل خادم يعمل بلا نافذة يُعد عملية في فضاء المستخدم؟ علّل.
3. كيف يمكن أن يعمل عشرون برنامجًا على جهاز يملك أربعة معالجات منطقية؟
4. تستخدم عمليتان العنوان الافتراضي نفسه. هل يعني ذلك أنهما تقرآن البيانات نفسها؟
5. ما الفرق الجوهري بين `fork()` وإحدى دوال عائلة `exec`؟ ماذا يحدث لمعرّف العملية في كل حالة؟
6. هل تشغيل برنامج باسم `root` يجعله ينفذ كل تعليماته في نمط النواة؟
7. لم تُفعّل مساحة مبادلة على جهازك. هل يعني ذلك عدم وجود ذاكرة افتراضية؟
8. لماذا يمكن أن يكون `uname -r` و`cat /etc/os-release` مفيدين معًا عند وصف مشكلة؟

إجابات مقترحة

1. المتصفح عملية أو مجموعة عمليات معزولة نسبيًا؛ تستطيع النواة إنهاء العملية المتعطلة دون أن تتوقف هي. لكن العملية قد تملك إذن تعديل ملفاتك، فلا يساوي عزل الذاكرة حماية البيانات من كل فعل مسموح.
2. نعم. «المستخدم» في اصطلاح فضاء المستخدم لا يشترط وجود تفاعل بشري مباشر أو نافذة. المقصود بيئة التنفيذ خارج فضاء النواة.
3. تمنح الجدولة المهام فرصًا للتنفيذ بالتناوب، ويمكن لأربع مهام مثلًا أن تنفذ بالتوازي على أربعة معالجات منطقية. كما أن كثيرًا من المهام ينتظر إدخالًا أو حدثًا ولا يحتاج إلى المعالج طوال الوقت.
4. لا. يُفسر العنوان في سياق مساحة عناوين كل عملية. قد يترجم إلى صفحتين ماديتين مختلفتين، ولا تحدث المشاركة إلا حين تُعد الخرائط لذلك.
5. ينشئ `fork()` عملية ابنة ذات معرّف جديد. أما `exec` فيستبدل البرنامج الجاري داخل العملية ويحافظ على معرّفها. الإنشاء والاستبدال مفهومان مختلفان حتى لو اجتمعا في مسار تشغيل واحد.
6. لا. تعمل تعليمات البرنامج العادية في نمط المستخدم، وتُستخدم نداءات النظام لطلب خدمات النواة. صلاحيات الحساب تؤثر في ما تسمح به النواة، ولا تلغي الفصل بين نمطي التنفيذ.

7. لا. الذاكرة الافتراضية آلية عناوين وخرائط وحماية؛ المبادلة وسيلة إضافية لإدارة بعض محتوى الذاكرة باستخدام تخزين مساند. يمكن استخدام الأولى دون الثانية.
8. يصف الأول النواة الجارية، ويصف الثاني التوزيعة أو بيئة نظام التشغيل التي يحتوي ملفها على تلك المعلومات. الجمع بينهما أدق من الاكتفاء باسم تجاري أو برقم واحد، خصوصًا في البيئات المعزولة.

خلاصة الفصل

- **التجريد** يساعدك على اختيار مستوى التفاصيل الذي يفسر المشكلة.
- **العتاد** يوفر الموارد، و**النواة** تنظم استخدامها، و**عمليات المستخدم** تنجز معظم الأعمال التي تهمك مباشرة.
- **الجدولة** تنظم التنفيذ، و**الذاكرة الافتراضية** تنظم العناوين والعزل والمشاركة.
- **نداءات النظام** طريق مضبوط لطلب خدمات النواة؛ لا يجعل استخدامها التطبيق جزءًا من النواة.
- **هوية المستخدم وصلاحياته** ليستا نمط تنفيذ المعالج. حتى `root` لا يساوي النواة.
- أفضل اختبار لفهمك أن تربط هذه الصورة بما شاهدته في `ps` و`free`و`id`، وأن تستطيع تفسير حدود كل ملاحظة.

ارجع إلى المعجم عند التباس المصطلحات، وإلى ملاحظات الإعداد للمصادر وحدود التبسيط. والخطوة التالية في المشروع هي إعداد الفصل الثاني: الأوامر الأساسية وبنية الأدلة.

معجم الدفعة الأولى

إضافة تعليمية من الإعداد العربي. هذه تعريفات تمهيدية مرتبطة بالفصل الأول، وليست تعريفات معيارية تستوعب كل الحالات. نلتزم بهذه المقابلات في الدفعات اللاحقة، ونوثّق أي تغيير اصطلاحي بدل تبديل الكلمات بصمت.

النظام والتنفيذ

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Abstraction	التجريد	التركيز على الوظيفة والعلاقات مع إرجاء تفاصيل لا يحتاج إليها السؤال الحالي.
Component	مكوّن	جزء من النظام يمكن وصف وظيفته وواجهاته مع الأجزاء الأخرى.
Layer	طبقة	مجموعة مكونات عند مستوى مناسب من التجريد، ليست كل طبقة حدًا أمنيًا.
Hardware	العتاد	الأجزاء المادية للحاسوب وأجهزته.
Operating system	نظام التشغيل	المنظومة التي تدير الموارد وتقدم بيئة وخدمات لتشغيل البرامج؛ نحدد نطاق المقصود بحسب السياق.
Distribution	توزيعة	تجميع للنواة والأدوات والمكتبات والخدمات، مع وسائل لتثبيتها وإدارتها.
Kernel	النواة	المكوّن المركزي الذي ينظم موارد النظام ويقدم واجهات محمية للعمليات.
CPU	وحدة المعالجة المركزية، المعالج	العتاد الذي ينفذ التعليمات.
CPU core	نواة معالجة	وحدة تنفيذ داخل المعالج، ليست نواة نظام التشغيل.
Logical CPU	معالج منطقي	مورد تنفيذ تراه النواة وتستطيع إسناد مهمة إليه، لا يساوي بالضرورة نواة مادية مستقلة.
Program	برنامج	تعليمات وبيانات معدّة للتنفيذ، وغالبًا تكون محفوظة في ملف.
Process	عملية	مثيل جارٍ من تنفيذ برنامج، له حالة وموارد ومعرّف.
Process ID / PID	معرّف العملية	رقم يميز العملية ضمن فضاء معرّفات العمليات الذي نلاحظها فيه.

Parent process	العملية الأم	العملية المرتبطة بعلاقة الأبوة بعملية ابنة؛ نستخدم نموذج الإنشاء المعتاد في هذا الفصل.
Child process	العملية الابنة	العملية الناتجة عن الإنشاء في نموذج الأم والابنة.
Thread	خيط تنفيذ	مسار تنفيذ داخل عملية؛ تشترك خيوط العملية في مساحة عناوينها.
Scheduling	الجدولة	اختيار المهام التي تعمل وتوقيت إتاحة المعالج لها وفق سياسة النظام.
Context switch	تبديل السياق	حفظ ما يلزم من سياق تنفيذ واستعادة سياق آخر لمتابعة مهمة مختلفة.
Time slice	شريحة زمنية	حصة زمنية في النموذج المبسط لتقاسم المعالج، ليست مدة ثابتة موحدة لكل المهام.
Concurrency	التزامن	تقدم أعمال متعددة خلال فترة واحدة، ولو بالتناوب.
Parallelism	التوازي	تنفيذ أعمال متعددة في الوقت نفسه على موارد تنفيذ متعددة.
Interrupt	مقاطعة	حدث يستدعي معالجة خاصة، مثل حدث مؤقت أو إشارة من جهاز.

الذاكرة والواجهات والصلاحيات

المصطلح الإنجليزي	المقابل العربي	المعنى في هذا الكتاب
Main memory / RAM	الذاكرة الرئيسية	الذاكرة التي تحتوي على التعليمات والبيانات اللازمة للعمل الجاري، وهي متطايرة عادة.
Persistent storage	التخزين الدائم	وسيط يحتفظ بالبيانات دون الاعتماد على بقاء الجهاز قيد التشغيل، مثل قرص التخزين.
Bit	بت	وحدة ثنائية تأخذ القيمة صفر أو واحد.
State	حالة	وصف مكوّن أو نظام عند لحظة معينة، على مستوى تفصيل نحدده.
Process image	صورة العملية	تمثيل البرنامج وبياناته في الذاكرة، ليست صورة رسومية.

Address space	فضاء العناوين	مجموعة العناوين التي يمكن تفسيرها في سياق عملية أو بيئة معينة.
Virtual memory	الذاكرة الافتراضية	نموذج يفصل العناوين التي يستخدمها البرنامج عن ترتيب الذاكرة المادي ويتيح تنظيمها وحمايتها.
MMU	وحدة إدارة الذاكرة	عتاد يساعد على ترجمة العناوين وتطبيق حماية الذاكرة.
Page table	جدول الصفحات	بنية تديرها النواة لوصف خرائط صفحات الذاكرة وخصائص الوصول إليها.
Swap	مساحة المبادلة؛ المبادلة	تخزين مساند يمكن استخدامه لبعض صفحات الذاكرة؛ ليس هو الذاكرة الافتراضية كلها.
Copy-on-Write	النسخ عند الكتابة	مشاركة مادية مؤقتة لصفحات، مع إنشاء نسخة مستقلة عندما تتطلب الكتابة ذلك.
User mode	نمط المستخدم	نمط تنفيذ محدود الامتياز لتعليمات برامج المستخدم.
Kernel mode	نمط النواة	نمط تنفيذ يتيح تعليمات وعمليات ذات امتياز لإدارة النظام.
User space / userland	فضاء المستخدم	بيئة عمليات المستخدم ومجالات ذاكرتها، بما فيها كثير من الخدمات الخلفية.
Kernel space	فضاء النواة	مجال الذاكرة المحمي الخاص بالنواة.
Device driver	برنامج تشغيل جهاز	برمجيات تعرف تفاصيل التعامل مع جهاز وتدمجه في واجهات النظام.
System call / syscall	نداء نظام	واجهة تطلب عبرها عملية خدمة من النواة.
Pseudo-device	جهاز زائف	واجهة برمجية شبيهة بواجهة جهاز، دون اشتراط وجود عتاد مستقل يقابلها.
User / UID	مستخدم / معرّف المستخدم	هوية ومرجع رقمي يستعملهما النظام في بيانات الاعتماد وقرارات الصلاحيات.
Superuser / root	المستخدم ذو الصلاحيات العليا	الحساب الإداري التقليدي ذو المعرّف صفر؛ ليس نواة النظام.

Group / GID	مجموعة / معرّف المجموعة	هوية جماعية تساعد على تنظيم أذونات الوصول والمشاركة.
Permissions	أذونات الوصول، الصلاحيات	قواعد تحدد العمليات المسموح بها على مورد أو ضمن سياق معين.
Capabilities	القدرات	وحدات امتياز تفصل بعض الصلاحيات الإدارية إلى عناصر يمكن ضبطها.
Namespace	فضاء أسماء	آلية تقدم للعمليات رؤية معزولة لمورد من موارد النظام، مثل معرّفات العمليات.
Shell	صدفة الأوامر	برنامج يفسر الأوامر ويوفر وسائل لتركيبها وتشغيلها.
Terminal	طرفية	واجهة لإدخال النص وعرضه؛ يمكن أن تعمل الصدفة داخلها.
Directory	دليل، مجلد	بنية تربط أسماء بملفات وأدلة أخرى ضمن شجرة نظام الملفات.
Log	سجل	رسائل أو بيانات تسجل أحداثًا للمراقبة والتشخيص.

فروق تستحق بطاقة تذكير

- **العملية ليست البرنامج:** يمكن أن ينتج تشغيل البرنامج نفسه عدة عمليات.
- **النواة ليست نواة المعالج:** الأولى برمجيات إدارة، والثانية عتاد تنفيذ.
- **فضاء المستخدم ليس حساب المستخدم:** الأول بيئة تنفيذ، والثاني هوية وصلاحيات.
- **المستخدم `root` ليس الدليل `/`:** الأول حساب، والثاني جذر شجرة الملفات.
- **الذاكرة الافتراضية ليست المبادلة:** يمكن أن توجد الأولى من دون تفعيل الثانية.
- **نداء النظام ليس أمر الصدفة:** قد يستخدم أمر واحد نداءات نظام كثيرة أثناء تنفيذه.
- **`fork` ليس `exec`:** الأول ينشئ عملية، والثاني يستبدل برنامج عملية موجودة.

ملاحظات الإعداد والمراجع

المصدر وحدود التغطية

المصدر هو **How Linux Works: What Every Superuser Should Know**، **الطبعة الثانية**، تأليف **Brian Ward**، نشر **No Starch Press**. نعتمد ملف `HowLinuxWorks.pdf` المرفق بالمستودع، وعدد صفحاته **338 صفحة**. تنص صفحة الحقوق، وهي الصفحة 337 من الملف، على حقوق الطبعة الثانية لعام 2015، كما توجد في الملف بيانات نشر من عام 2014.

لا تُشتق إحالات الصفحات هنا من العلامات المرجعية المضمنة في PDF؛ بعض هذه العلامات لا يطابق بداية النص بدقة. جرى الاعتماد على مواضع العناوين في الصفحات نفسها. يعرض ملف `book/manifest.json` نطاقات المصدر وحالة كل فصل، بما فيها الفصول غير المعدّة بعد.

مادة هذه الدفعة	موضعها في ملف المصدر	طريقة التعامل معها
المقدمة العربية	المقدمة، الصفحات من 4 إلى 6	إعادة صياغة لأهداف التعلم والجمهور وطريقة القراءة والتنظيم، مع إرشادات عربية إضافية.
الفصل الأول	الصفحات من 8 إلى 14	إعداد تعليمي لجميع أقسام الفصل من 1.1 إلى 1.6، مع الحفاظ على ترتيبها.
الرسوم الثلاثة	أفكار الأشكال 1-1 و 1-2 و1-3	مخططات جديدة بالعربية؛ ليست استنساخًا للتصميم أو لجميع أمثلة الرسوم الأصلية.
المختبر والأسئلة والإجابات والمعجم	إضافات إلى الفصل	محتوى تعليمي مضاف، وليس ترجمة لتمارين أو معجم في الأصل.
التوضيحات التقنية	مراجع أدناه	تفصل التبسيط التمهيدي عن تفاصيل مهمة في تطبيقات لينكس.

لم تتضمن الدفعة ترجمة عبارات الثناء، أو صفحة شكر المؤلف، أو الفصول من الثاني إلى السابع عشر، أو قائمة مراجع الأصل وفهرسه ونبذة مؤلفه. ذكر أسماء تلك المواد في خريطة المشروع **لا يعني إعداد محتواها**.

مراجع التوضيحات المضافة

المراجع الآتية صفحات توثيق خارجية يمكن الرجوع إليها للتثبت أو التوسع. الرقم بين القوسين في اسم صفحة دليل يحدد قسمها. استُخدمت هذه المراجع للتحقق من التوضيحات ذات الصلة أثناء إعداد هذه المسودة بتاريخ 7 **سبتمبر 2026**؛ لا يعني ذلك مراجعة بشرية متخصصة للدفعة كلها.

1. صفحة الدليل fork(2): إنشاء العملية الابنة، واستقلال فضاءي العناوين، والنسخ عند الكتابة، والتمييز بين واجهة المكتبة والتنفيذ الداخلي. ترتبط بالقسم 1.3.4.
2. صفحة الدليل execve(2): استبدال صورة البرنامج داخل العملية، والسمات التي تبقى أو تتغير. ترتبط بالقسم 1.3.4.
3. صفحة الدليل clone(2): واجهات إنشاء المهام والتحكم في الموارد المشتركة، ومنها `clone3()`. تبين حدود تعميم نموذج `fork` وحده.
4. صفحة الدليل pthreads(7): تعدد خيوط التنفيذ داخل العملية والموارد التي تتشاركها. ترتبط بتوضيح الجدولة في القسم 1.3.1.
5. صفحة الدليل capabilities(7): تجزئة الامتيازات الإدارية إلى قدرات. ترتبط بالقسم 1.5.
6. صفحة الدليل namespaces(7): الرؤية المعزولة للموارد وصلتها بالحاويات. ترتبط بالمقدمة والقسم 1.5 وحدود ملاحظة العمليات في المختبر.
7. صفحة الدليل free(1): معنى أعمدة الذاكرة، ولا سيما `available`، وخيار العرض المقروء. ترتبط بتجربة الذاكرة.
8. صفحة الدليل systemd-journald.service(8): وظيفة خدمة جمع السجلات ضمن منظومة `systemd`. ترتبط بالمثال المضاف في القسم 1.4.

تصف صفحات الدليل حالات تفصيلية أكثر من هذا الفصل. لا يلزم إتقانها كلها الآن، ولا ينبغي تحويل النموذج المبسط هنا إلى ضمان مطلق بشأن كل معمارية أو توزيعة أو إعداد أمني.

سياسة الإعداد والمراجعة

- نكتب بالفصحى، ونشرح المصطلح عند ظهوره الأول، ونثبت مقابله في المعجم.
- نحافظ على أسماء الأوامر والمسارات والخيارات والمعرّفات. لا نترجم محتوى يُفترض أن يفسّره برنامج.
- نميز الأوامر عن المخرجات، ونصرّح بأن المخرجات التوضيحية قد تختلف عن جهاز القارئ.
- نحافظ على أرقام أقسام الأصل. الإضافات التعليمية لا تحصل على أرقام توحي بأنها أقسام أصلية جديدة.
- نوسم التحديث والتصحيح والتوسع بدل نسبته إلى المؤلف أو تحديث الطبعة بصمت.
- لا نضع نصًا إنجليزيًا غير مُعدّ داخل فصل ثم نعلن اكتماله. المواد غير المنجزة تبقى ظاهرة بوصفها «لم يبدأ إعدادها».
- الاختبارات الآلية تتحقق من بنية الملفات والروابط وبناء المخرجات وبعض الأمثلة؛ وهي لا تحل محل مراجعة عربية وتقنية بشرية.

للمراجعة، اذكر اسم الملف وعنوان الفقرة، ثم اقترح التعديل وسببه. فرّق بين خطأ تقني، وخيار اصطلاحي، وتحسين أسلوبي. الهدف أن يتمكن القارئ من فهم الفكرة وتجربتها بأمان، لا أن تبدو العبارة قريبة من ترتيب الكلمات الإنجليزية فقط.

خريطة إنجاز الكتاب

المتاح: مسودة 1 من أصل 17 فصلًا، إضافة إلى المقدمة العربية. هذه أعداد فصول، وليست نسبة مئوية من كلمات الكتاب أو صفحاته. «مسودة متاحة» تعني وجود نص قابل للقراءة والمراجعة، لا اعتماد ترجمة نهائية.

الفصل	العنوان العربي المقترح	صفحات ملف المصدر	الحالة
1	الصورة العامة	8–14	مسودة متاحة
2	الأوامر الأساسية وبنية الأدلة	15–41	لم يبدأ إعدادها
3	الأجهزة	42–58	لم يبدأ إعدادها
4	الأقراص وأنظمة الملفات	59–80	لم يبدأ إعدادها
5	كيف تُقلع نواة لينكس	81–94	لم يبدأ إعدادها
6	كيف يبدأ فضاء المستخدم	95–124	لم يبدأ إعدادها
7	إعداد النظام: السجلات والوقت والمهام الدفعية والمستخدمون	125–142	لم يبدأ إعدادها
8	نظرة أعمق إلى العمليات واستخدام الموارد	143–158	لم يبدأ إعدادها
9	فهم الشبكة وإعدادها	159–191	لم يبدأ إعدادها
10	تطبيقات الشبكة وخدماتها	192–209	لم يبدأ إعدادها
11	مقدمة إلى سكربتات الصدفة	210–228	لم يبدأ إعدادها
12	نقل الملفات عبر الشبكة	229–241	لم يبدأ إعدادها
13	بيئات المستخدمين	241–249	لم يبدأ إعدادها
14	جولة موجزة في سطح مكتب لينكس	250–259	لم يبدأ إعدادها
15	أدوات التطوير	260–275	لم يبدأ إعدادها

16	مقدمة إلى بناء البرامج من شيفرة C المصدرية	276–287	لم يبدأ إعدادها
17	البناء على الأساسيات	288–291	لم يبدأ إعدادها

تبدأ بعض الفصول في الصفحة نفسها التي ينتهي فيها الفصل السابق؛ مثل الفصلين 12 و13 في الصفحة 241. لذلك قد تتداخل حدود الصفحات دون تداخل المحتوى.

مواد أخرى لم تُعد بعد: عبارات الثناء، شكر المؤلف، قائمة مراجع الأصل، فهرس الأصل، نبذة عن المؤلف، صفحة الحقوق الأصلية.

الخطوة التالية: إعداد الفصل الثاني، مع مراجعة أمثلته على توزيعة معاصرة، والحفاظ على فصل التوضيحات المضافة عن مادة الأصل. لا يحتوي هذا الإصدار على صفحات فارغة أو ملخصات بديلة تدّعي أنها الفصول المتبقية.